

CSCI 2800 — Computer Architecture and Operating Systems (CAOS)
Homework 2 (document version 1.0)
Dynamic Memory Allocation, Processes, and Named Pipes

- This homework is due in Submitty by 10:00PM Thursday, October 15, 2026
- You can use at most **two late days** on this assignment
- This homework is to be done individually, so **please do not share your code with others**
- Please do **not** use AI to assist you on this homework
- Place all of your code in `hw2.c` for submission; no other files may be included
- You **must** use C for this assignment, and all submitted code **must** successfully compile via `gcc` with no warning messages when the `-Wall` (i.e., warn all) compiler option is used; we will also use `-Werror`, which will treat all warnings as critical errors
- All submitted code **must** successfully compile and run on Submitty, which currently uses Ubuntu v22.04.5 LTS and `gcc` version 11.4.0 (`Ubuntu 11.4.0-1ubuntu1~22.04.3`)
- This assignment will be entirely auto-graded by Submitty; you will have **eight** penalty-free submissions, after which points will slowly be deducted, e.g., -1 on submission #9, etc.
- You will have at least **three** days before the due date to start submitting your code to Submitty; if auto-grading is not available three days before the due date, the due date will be 10:00PM three days after auto-grading becomes available

Hints and reminders

Do **not** rely on program output to confirm whether your code is correct. Consistently allocate **exactly** the number of bytes you need regardless of whether you use static or dynamic memory allocation. Further, deallocate dynamically allocated memory via `free()` at the earliest possible point in your code.

Make use of `valgrind` (or `drmemory` or other dynamic memory checkers) or the “valgrind unlimited” gradeable in Submitty to check for errors with dynamic memory allocation and dynamic memory usage. As another helpful hint, close open file descriptors as soon as you are done using them.

Always read (and re-read!) the `man` pages for library functions (section 3), system calls (section 2), etc., especially if you do not recognize a function you are required to use.

To understand how `man` pages are organized, check out the `man` page for `man` itself (via `man man`).

Homework specifications

In this second homework, you will use C to implement a rudimentary spell-checking program that performs its spell-check operations **in parallel** using multiple child processes.

Given a dictionary file of valid words, you first construct a *Trie* (pronounced “try”) data structure, which is a multi-layer structure described on the next page. Once you have processed the input file and constructed the Trie, you then are to read candidate words from a named pipe, i.e., a **fifo**. If the candidate word is valid, echo back the word on a second **fifo**; if the candidate word is not valid, send a suggested word on the second **fifo**.

Your program should cleanly shut down when the two **fifos** are closed.

No square brackets allowed

To emphasize the use of pointers and pointer arithmetic, **you are not allowed to use square brackets** anywhere in your code! If a '[' or ']' character is detected in your code, **including within comments**, Submittity will remove that entire line from your code before running `gcc`.

Command-line arguments and memory allocation

The first command-line argument specifies the name of the dictionary file (e.g., `words.txt`). This file contains one or more valid words, each word on its own line; this means, each word is delimited by a newline '\n' character. To be a valid word, a word must be alpha (i.e., `a-z` and/or `A-Z`) characters and be of a non-zero length up to at most 16 characters.

Note that the use of `FILE` is not allowed; instead, use file descriptors and the `open()`, `read()`, and `close()` system calls.

The second command-line argument specifies the name of the **fifo** to use to read candidate words. A separate process that is hidden within Submittity will send candidate words on this pipe, each one delimited with a hyphen '-' character.

Since pipes and **fifos** are unidirectional communication channels, the third command-line argument specifies the name of another **fifo** on which you will return the result of your spell-check operation.

Be sure to open the first **fifo** (for reading) before you open the second **fifo** (for writing). This will match the hidden test process and avoid a “deadlock” in which both processes are blocked on `open()` calls.

If you do not receive the correct number of arguments, display the following message on `stderr` by using `fprintf()`: `USAGE: hw2.out <dictionary-file> <read-fifo> <write-fifo>`

The Trie data structure

Often used for spell-checking and various prefix-matching applications, the Trie data structure is a multi-layer tree structure that stores valid words by encoding each word character by character going left to right.

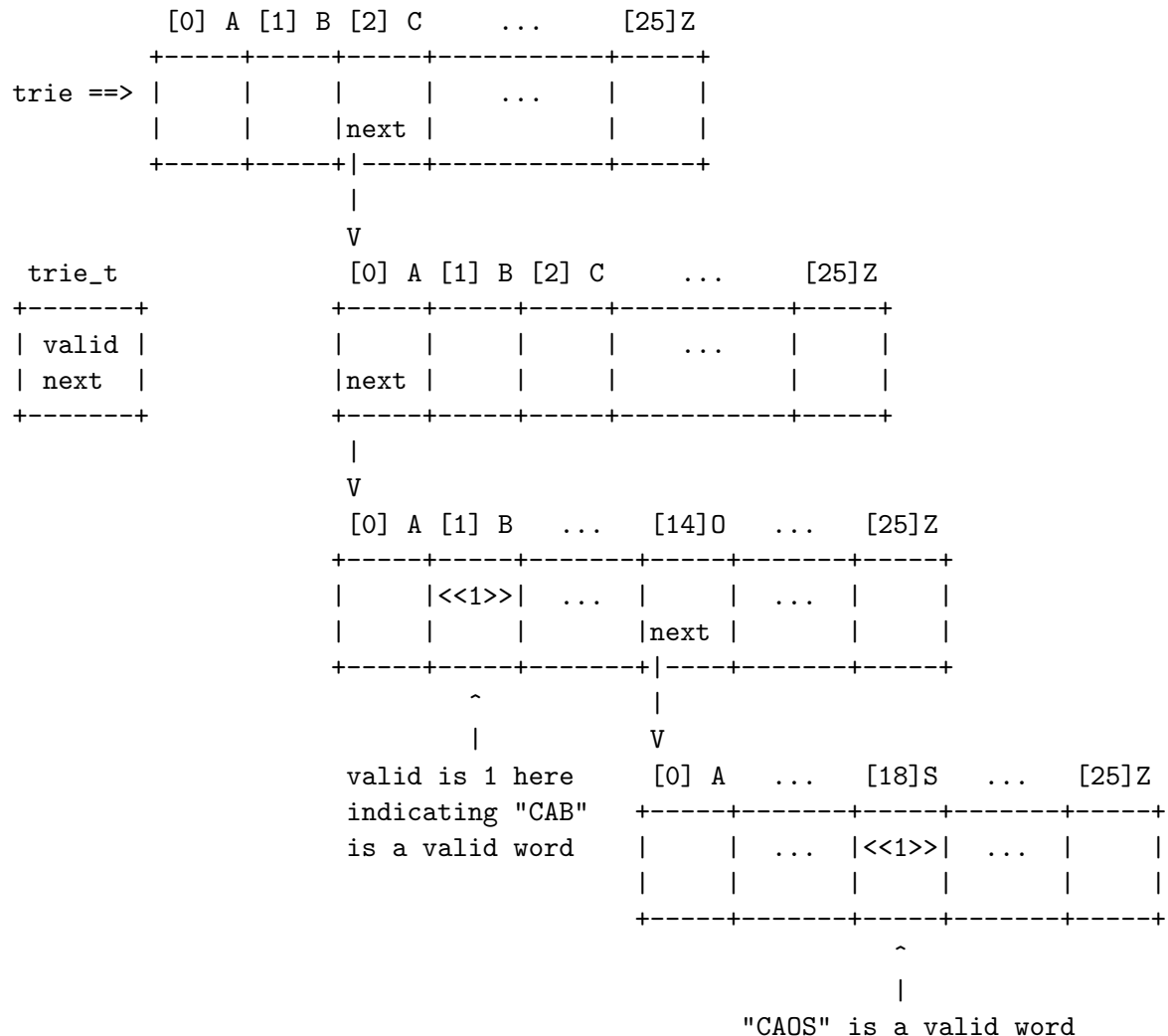
At each level, a 26-element array is mapped to each letter of the alphabet. We will use uppercase characters **A-Z** only, so convert all dictionary words to uppercase before storing the word in the Trie. To map 'A' to index 0, 'B' to index 1, and so on, remember that a **char** data type is simply a numeric data type.

Each 26-element array contains 26 “structures” (i.e., a **struct** in C), each of which contains a true/false one-byte **char** indicating if we have reached a valid word and a pointer to the next 26-element array in the Trie if there is another level. The **struct** to use is shown below; specifically, this defines the **struct** as a data type called **trie_t**.

```
typedef struct trie
{
    char valid;           /* 0 if not a valid word to this point; 1 if word is valid */
    struct trie * next; /* pointer to next level of Trie; or NULL if no next level */
}
trie_t;
```

The overall Trie data structure is shown in an example on the next page.

The Trie data structure is shown below; here, valid words are "CAOS" and "CAB". All other blank spaces are instances of the `trie struct` with 0 for `valid` and `NULL` for `next`.



Start with the given `hw2-start.c` code posted in Submitty. This file provides the `trie struct` definition given above, as well as a debugging function called `print_trie()` that prints all elements of the Trie once it is constructed. Use this function to better understand the Trie data structure.

Next, create other functions for the Trie (e.g., `add_word()`, `is_valid_word()`, etc.) based on the `print_trie()` function. Only add what you need—this is **not** a Data Structures class!

Given the importance of 0 and `NULL` in the Trie data structure (i.e., to indicate no valid word and no next level in the Trie), be sure to use `calloc()` for your dynamic memory allocations. As a hint, your first layer would be constructed as follows:

```

/* create array to hold top layer of the Trie */
trie_t * trie = calloc( 26, sizeof( trie_t ) );
if ( trie == NULL ) { perror( "calloc() failed" ); return EXIT_FAILURE; }

```

As with Homework 1, do **not** use `malloc()` or `memset()` in your code.

Protocol for the named pipes

Once you have constructed your Trie of valid words, open each `fifo` by using the `open()` system call. Remember that a `fifo` is entirely in memory; we simply use the file name to access this in-memory buffer. Aside from using `open()` here, a `fifo` behaves the same way a pipe does, i.e., use `read()`, `write()`, and `close()` accordingly.

For your inter-process protocol, you should receive a candidate word on the first `fifo`, delimited by a hyphen '-' character. Be prepared to receive multiple such requests in rapid succession.

For each candidate word, call `fork()` to hand the spell-check subtask off to a child process. The child process must write its results to the second `fifo`. Fortunately, the file descriptor (fd) table is inherited by the child process when you call `fork()`.

The child process must write to the second `fifo` as follows:

- If the candidate word is valid, echo the word back on the `fifo`, ending it with a period '.' character. For example, if the candidate word is "CAOS" and "CAOS" is a valid word, write "CAOS." to the second `fifo`.
- If the candidate word is not valid, use the Trie to find a suggested alternate (see below), then write the candidate word and suggested valid word, using a '>' character to delimit the two words; end this with '?'. For example, if the candidate word is "CAOF", write "CAOF>CAOS?" to the second `fifo`.

When you write to the `fifo`, be sure to write your response using **exactly one** `write()` call. Otherwise, multiple child processes could accidentally interleave their responses.

Back in the parent process, be sure to call `waitpid()` to acknowledge all terminated child processes. Remember this is required or else the “zombie” child process will live on forever.

Finding an alternate word

For a candidate word that is not valid, use the following algorithm to find an alternate word to suggest. Starting from the index of the last letter of the candidate word (e.g., 'F' in "CAOF"), look for valid words at subsequent left-to-right index positions along the same Trie layer (e.g., "CAOG", "CAOH", etc.) until you find a valid word. If you reach 'Z', circle back to 'A' at that same layer, continuing until you reach your starting point (e.g., 'F'). In other words, treat this as a circular array. Given the structure of Trie, there will always be an alternate in this case.

If you did not reach the last letter of the candidate word, this means there was not a layer in the Trie that went deep enough. In this case, use the “closest” word at the deepest layer that has a valid word (along the path of the candidate word); note that “closest” here means you apply the left-to-right circular array approach from above.

If no valid words exist at any layer for the given candidate word, offer no suggested alternate by omitting the '>' character and leaving the second part of your response blank, e.g., "FOCS?\n".

Required output

Your program must display a line of output whenever a new layer is added to the Trie. (Do not display a line of output for the top layer.) Once the Trie is complete, display a line of output reporting the total number of valid words in the Trie.

Below is an example that shows the output format you must follow based on the sample input file shown.

```
bash$ cat input.txt
AA
AAH
AAHED
AAHING
AAHS
AAL
AALII
AALIIS
AALS
AARDVARK
CAOS
bash$ ./hw2.out input.txt /tmp/fifo-read.txt /tmp/fifo-write.txt
PARENT: creating new Trie layer for 'A' of "AA"
PARENT: creating new Trie layer for 'A' of "AAH"
PARENT: creating new Trie layer for 'H' of "AAHED"
PARENT: creating new Trie layer for 'E' of "AAHED"
PARENT: creating new Trie layer for 'I' of "AAHING"
PARENT: creating new Trie layer for 'N' of "AAHING"
PARENT: creating new Trie layer for 'L' of "AALII"
PARENT: creating new Trie layer for 'I' of "AALII"
PARENT: creating new Trie layer for 'I' of "AALIIS"
PARENT: creating new Trie layer for 'R' of "AARDVARK"
PARENT: creating new Trie layer for 'D' of "AARDVARK"
PARENT: creating new Trie layer for 'V' of "AARDVARK"
PARENT: creating new Trie layer for 'A' of "AARDVARK"
PARENT: creating new Trie layer for 'R' of "AARDVARK"
PARENT: creating new Trie layer for 'C' of "CAOS"
PARENT: creating new Trie layer for 'A' of "CAOS"
PARENT: creating new Trie layer for 'O' of "CAOS"
PARENT: added 11 words to the Trie
bash$
```

No other output is required; however, adding debugging output is recommended.

Error handling

If improper command-line arguments are given, report an error message to `stderr` and exit the program with `EXIT_FAILURE`. In general, if an error is encountered, display a meaningful error message on `stderr` by using either `perror()` or `fprintf()`, then exit the program with `EXIT_FAILURE`. Only use `perror()` if the given library or system call sets the global `errno` variable.

Error messages must be one line only and use the following format:

```
ERROR: <error-text-here>
```

Submittity may check for the existence of the line above by verifying that the line starts with the "ERROR:" string.

Submission instructions

To submit your assignment and perform **final** testing of your code, we will use the auto-grading features of Submittity. Your code will not be manually reviewed.

To help ensure that your program executes properly, consider making use of the `DEBUG_MODE` pre-processor technique shown below. This helps to avoid accidentally displaying extraneous output in Submittity. Here is an example:

```
#ifdef DEBUG_MODE
    printf( "the value of x is %d and y is %f\n", x, y );
    printf( "here123\n" );
    printf( "why is my program crashing here?!\n" );
#endif
```

To compile this code in “debug” mode, use the `-D` flag as follows:

```
bash$ gcc -Wall -Werror -g -D DEBUG_MODE hw2.c
```

Submittity will use the following to compile your code:

```
bash$ gcc -Wall -Werror hw2.c
```

Also, Submittity uses buffered output when writing data; therefore, add the line below as the first statement in your `main()` function to have all output sent to `stdout` be non-buffered. If your program hangs or crashes, this will ensure that you will see output to that point.

```
setvbuf( stdout, NULL, _IONBF, 0 );
```